

CS247 Assignment 2 — Marching Squares (2D iso-contours)

How to run

```
cd Assignment_2
cmake -S . -B build
cmake --build build -j
cd build && ./assignment
```

Two windows open. The “Marching Squares” window is this assignment (the 3D window is for A3, it just shows the bounding box here).

Key	Action
1 / 2	load lobster / head
A	switch axis
W / S	next / previous slice
I / 0	iso value +0.01 / +0.1
K / L	iso value -0.01 / -0.1
C	toggle contour
Esc	quit

What it does

Same slice viewer as A1, but now I overlay a 2D iso-contour. The contour is computed on the CPU with Marching Squares and drawn as `GL_LINES` on top of the slice. It only recomputes when the slice / axis / iso changes, not every frame — every relevant key handler calls `marchingSquares()`.

Implementation

Per cell. For each cell of the current slice I sample the four corner values (BL, BR, TR, TL) and classify them against the iso value into a 4-bit case index:

```
if (v0 > iso + eps) caseIdx |= 0x1; // BL
if (v1 > iso + eps) caseIdx |= 0x2; // BR
if (v2 > iso + eps) caseIdx |= 0x4; // TR
if (v3 > iso + eps) caseIdx |= 0x8; // TL
```

`edgeTable2D[caseIdx]` tells me which edges are crossed.

Exact position by interpolation. For each crossed edge I interpolate $t = (iso - v_A) / (v_B - v_A)$ so the point sits exactly where the iso value crosses, not at the midpoint. Points use voxel-centre NDC $(2i+1)/dim - 1$ so they line up with the slice texture.

Epsilon. `eps = 1e-3` is used twice: classifying corners as $> iso + eps$ (so a corner sitting exactly on the iso never lands a point on it), and guarding the interpolation denominator ($t = 0.5$ if $|v_B - v_A| < eps$) so I never divide by ~ 0 .

Ambiguous cases. For case 5 and 10 (opposite corners above) all four edges cross. I wrap each above-corner with its own arc — simple and deterministic.

Drawing. Contour points go into a `std::vector<float>`, uploaded to a `GL_STREAM_DRAW` VBO and drawn with `glDrawArrays(GL_LINES, ...)`. The contour reuses the slice shader with `useTexture = 0` and an orange `vertexColor`, plus the same `model matrix`, so it overlays correctly at any window size.

Results

See `demon.mov` — iso sweeps, axis cycling, slice scrubbing, and toggling the overlay. The orange contour matches the reference screenshot.